Projet de chasse multi-agent - Dedale

Groupe 11 Alexandre Copin Gildas De Michiel

Année : 2025–2026

Table des matières

1 Introduction **2**

2 Architectures générale du code **3**
2.1 Diagramme UML Simplifié 3
2.2 Caractéristique de l'agent 3

3 Présentation de nos choix d'implémentation **5**
3.1 Exploration 5
3.2 Communication 6
3.3 Chasse 8
3.4 Mur 9

4 Conclusion **10**

1 Introduction

Le but de notre projet consiste à développer un système multi-agent inspiré du jeu «Hunt the Wumpus (Gregory Yob, 1972)». Le projet est composé de deux objectifs. Le premier objectif est de concevoir des agents autonomes capables d'explorer un environnement inconnu. Le deuxième objectif est de localiser un ou plusieurs Golems, puis de les capturer en les bloquant sur des nœuds du graphe.

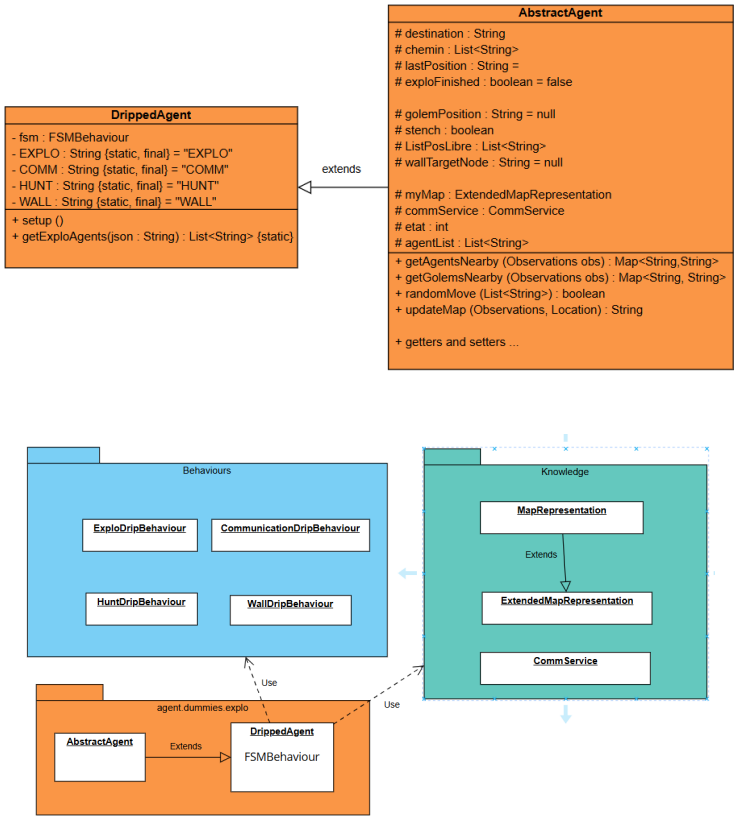
Dans ce projet, nous avons utilisé la plateforme multi-agent JADE avec l'environnement Dedale. Les agents disposent de capacités limitées. En effet, ils ne possèdent qu'une communication limitée et des perceptions partielles. Ces limitations soulèvent de nombreuses difficultés qui seront résolues par la coordination et la coopération de nos agents.

Chaque agent combine des comportements d'exploration, de communication et de chasse au sein d'une architecture basée sur un automate à états finis (FSM).

Pour finir, ce rapport présente les choix techniques réalisés, les algorithmes mis en œuvre ainsi que les protocoles de communication utilisés. Nous nous attarderons aussi sur l'analyse des performances du système en termes de complexité (temps, mémoire, communication), les limites de notre approche, ainsi que les pistes d'amélioration possibles.

2 Architectures générale du code

2.1 Diagramme UML Simplifié



2.2 Caractéristique de l'agent

Dans notre implémentation, tous les agents possèdent les mêmes comportements. En effet, le comportement de chaque agent est structuré autour d'un automate à états finis (FSM) composé de quatre états principaux.

Le premier état est l'exploration qui consiste à explorer le graphe en découvrant de nouveaux nœuds non visités. Si tout les noeuds ont étés visités, alors il ira sur les plus anciens noeuds qu'il n'a pas visité. Si une odeur est detectée ou la position du golem est connu, l'agent passe dans l'état de chasse.

Le deuxième état est la communication. Cet état sert à recevoir et envoyer diverses informations des autres agents. Un agent peut notamment recevoir la carte d'un autre agent, ainsi que des informations sur la position du Golem. Il peut envoyer des informations pour aider les autres agents. Par défaut, chaque

autre état va vers la communication afin de traiter les messages reçu, puis on retourne dans l'état précédent (sauf changement par un traitement).

Le troisième état est la chasse. Pendant la chasse, le but de l'agent est de se rapprocher vers un Golem. Par défaut, l'agent continue dans l'état de communication. Cependant, s'il détecte le Golem autour de lui ou s'il a reçu sa position depuis l'état de communication, il continuera dans l'état mur. De plus, lorsqu'il détecte la position du Golem, il l'envoie à tous les agents proches. Enfin, si l'agent ne connaît pas la position du Golem et ne détecte pas d'odeur, alors il retourne dans l'état d'exploration.

Le quatrième et dernier état est le comportement mur. Une fois que l'agent est à côté d'un Golem, il reste sur place pour le bloquer. Pendant cet état, l'agent peut aussi envoyer la position du Golem, mais aussi les positions nécessaires pour bloquer le Golem. S'il bloque le chemin d'un autre agent vers une position nécessaire, alors uniquement ainsi il se déplacera. Par défaut, l'agent continue en communication pour recevoir des informations d'autre agent. Si le Golem n'est plus visible par l'agent alors il retourne en chasse pour le poursuivre.

Pour finir, l'agent possède de nombreuses connaissances permettant de réaliser plus facilement ses différents objectifs.

Tout d'abord l'agent possède des connaissances liées à son déplacement. En effet, l'agent connaît son nœud de destination, son trajet pour atteindre ce nœud, ainsi le nœud qu'il occupait précédemment.

Ensuite, pour faciliter l'encerclement du Golem, l'agent peut garder en mémoire la position du Golem, la présence ou non de l'odeur du Golem, une position libre qui permet de bloquer le Golem ainsi qu'une position où on lui demande d'aller bloquer.

De plus, l'agent possède des informations pour lui permettre de communiquer avec d'autres agents. L'agent garde en mémoire la liste des agents présents sur la carte, mais aussi les informations liées aux différents protocoles de communication.

Enfin, l'agent garde en mémoire son état précédent, possède une représentation de la topologie du graphe améliorer avec un timer sur les noeuds, un répertoire des différents message qu'il peut envoyer ainsi que savoir s'il a fini d'explorer le graphe.

3 Présentation de nos choix d'implémentation

3.1 Exploration

L'exploration a pour objectif de permettre aux agents de découvrir progressivement la topologie du graphe.

Le principe de notre exploration est le suivant. L'agent identifie les nœuds ouverts, puis sélectionne le plus proche grâce à la méthode *getShortestPathToClosestOpenNode()*. Cet méthode parcourt la carte de l'agent pour déterminer le nœud ouvert le plus proche. Ensuite, le trajet vers ce nœud est calculé avec l'algorithme Dijkstra. Une fois que le trajet stocké en mémoire, il est modifié à chaque déplacement.

Avant de se déplacer, l'agent vérifie si un autre agent occupe le nœud. Si c'est le cas alors il recalcule son chemin en évitant le nœud bloqué via la méthode *getShortestPathWithBlockedNodes()*. Si un tel chemin n'existe pas, alors il choisit un nœud adjacent au hasard.

Lors de son exploration, si l'agent détecte l'odeur d'un Golem, alors il passe dans l'état de chasse. De plus, s'il voit un Golem, il avertit les agents proches.

Une fois que l'agent a fini d'explorer la carte, il retourne vers les nœuds les plus anciens. Ainsi, il explore la carte dans le but de trouver un Golem.

Les déplacements en exploration reposent principalement sur l'utilisation de l'algorithme Dijkstra. Ainsi, si l'agent possède une carte avec $|V|$ nœuds et $|E|$ arêtes, le calcul du plus court chemin prend $O(|E| + |V|log(|V|))$. Or, si la carte possède $|S|$ nœuds ouverts, l'opération va être effectué $|S|$ fois. Trouver, le nœud ouvert le plus proche prend $|S|$. De plus, calculer les nœuds ouverts prend $|V|$. Aussi, le chemin le plus court vers le nœud ouvert le plus proche est calculer donc encore $O(|E| + |V|log(|V|))$. Au final, la complexité en temps est en :
$O(|V| + |S| \times (|E| + |V|log(|V|) + |E| + |V|log(|V|)$
$= O(|V| + (|S| + 1) \times (|E| + |V|log(|V|))$

L'algorithme n'est pas optimal pour l'exploration collective du graphe. En effet, plusieurs agents peuvent explorer les mêmes zones. Aussi, le coût computationnel en temps est élevé à cause du calcul fréquent de plus court chemin.

Cependant, il est efficace pour réduire les déplacements de l'agent. De plus, chaque agent peut modifier dynamiquement son trajet pour éviter les blocages.

3.2 Communication

La communication a pour but de faciliter et optimiser les échanges entre les agents. Plus particulièrement, elle permet de traiter les informations reçues par d'autres agents.

Notre communication repose sur deux composantes principales. La première composante est un service de communication qui permet à chaque agent d'envoyer des messages. Ce service est implémenté en tant que connaissances de l'agent. Ainsi, l'envoi d'informations n'est pas seulement limité à l'état de communication. La deuxième composante est l'état de communication. Cet état s'occupe uniquement de traiter et répondre aux messages reçus.

Les protocoles de communication reposent sur plusieurs types de messages basés sur les ACLMessage de JADE.

Le premier types de messages sont les messages de types *REQUEST*. Ces messages permettent de transmettre des informations aux autres agents. Périodiquement, les agents envoient un message avec le protocole *Ping* pour détecter les agents proches. Si un agent reçoit un message *Ping*, alors il répond avec un message *Pong*. Ce mécanisme permet d'établir la présence d'un agent pour ensuite commencer l'envoi d'informations. Un agent reçoit le protocole *WALL-NEEDED*, si un autre agent lui informe qu'un Golem est en train d'être encerclé. Ce message contient la position du Golem, les nœuds autour du Golem, ainsi que les nœuds autour du Golem qui ne sont pas encore occupés. Un agent reçoit le protocole *MOVE-ASIDE* s'il bloque un agent qui souhaite suivre le protocal *WALL-NEEDED*. Ce message contient la position où l'agent souhaite bloquer mais il n'affecte que les agents en mode mur. Les agents murs traitent se message en définissant une nouvelle zone accessible où ils peuvent se déplacer afin de continuer à bloquer le golem tout en laissant la place.

Le deuxième type de messages sont les messages de types *INFORM*. Ces messages permettent de recevoir des informations d'autres agents. Un agent reçoit le protocole *PONG*, si un autre agent a confirmé sa présence pour un échange d'informations. L'agent s'assure qu'il n'a pas envoyé sa carte récemment. Si assez de temps s'est écoulé, on peut envoyer la carte, sinon on ignore le message. Un agent reçoit le protocole *SHARE-TOPO*, si un autre agent lui a envoyé sa carte. Après la réception du message, l'agent complète sa propre carte avec les nouvelles informations reçues. Un agent reçoit le protocole *WUMPUS-LOCATION*, si un agent essaie de lui communiquer la position d'un Golem.

Le trosième type de messages sont les messages de types *PROPAGATE*. Ces messages permettent de diffuser une information sur un Golem. Si un agent reçoit un protocole *WUMPUS-ALERT* alors il informe les autres agents de la position du Golem. Chaque agent va à son tour diffuser la position du Golem à tous les agents proche de lui.

Les communications se font avec tous les agents. Soit n le nombre d'agents. Pour le partage de carte, chaque envoi de *PING* parcourt la liste des agents donc n messages. Chaque *PING* peut générer un *PONG* donc encore n messages.

Chaque *PONG* peut générer un envoi de carte. Ainsi, au maximum un agent peut répondre à n messages. D'où la complexité en nombre de messages est en $O(n + n + n) = O(n)$ pour un seul agent.

Pour les messages de propagation, chaque agent envoie au agent alentour sauf à l'expéditeur. Donc, la complexité en message est $O(n \times (n-1)) = O(n^2)$

Un agent gère au maximum 10 messages par cycle. Ceci permet d'éviter le blocage de l'agent. Un *PING* est envoyé toutes les 5 secondes, tandis qu'un partage de carte ne peut se faire que toutes les 15 secondes. De plus, après avoir reçu un *PONG*, l'agent attend 2 secondes. Ces implémentations permettent de limiter les échanges inutiles et éviter d'envoyer trop de messages.

3.3 Chasse

Le comportement de chasse est activé dès qu'un agent détecte une odeur ou reçoit une information indiquant la présence d'un Golem. Le but de cet état est de localiser le Golem.

Avant chaque déplacement, l'agent observe son voisinage. Ainsi, il identifie les nœuds libres, les nœuds libres contenant une odeur, et la présence ou non d'un Golem. Lorsqu'un Golem est repéré, l'agent enregistre sa position, la diffuse aux agents proches pour au final passer dans l'état mur. Si l'agent ne voit pas le Golem, mais a reçu un signal de blocage, alors il se dirige vers la position qu'on lui a indiquée.

Si aucune odeur n'est détectée ou si la position du Golem n'est pas connue, alors l'agent retourne dans l'état d'exploration.

Si l'agent connaît la position du Golem, mais n'est pas autour du Golem, alors il possède plusieurs comportements. Si la position du Golem, n'existe pas dans son graphe, alors il abandonne la poursuite et se dirige vers l'odeur la plus proche. Si l'agent a atteint la position du Golem, mais qu'il n'est plus présent et qu'il n'y a plus d'odeur, alors on retourne en exploration. Finalement, si aucun de ces événements ne s'est produit, alors on recalcule le chemin pour atteindre le Golem. Ce dernier cas permet de détecter le déplacement du Golem.

Pour finir, si le golem n'est pas localisé mais qu'une odeur est détecté, alors l'agent se dirige vers un voisin contenant une odeur. Il évite de revenir immédiatement sur sa position précédente pour éviter de rester bloqué dans une boucle.

Les déplacements en chasse reposent uniquement sur l'algorithme de Dijkstra. Or, on a déjà convenu que la complexité en temps de cet algorithme est en $O(|E| + |V| \times log(|V|))$ avec V les nœuds du graphe et E les arêtes du graphe. Pour ce qui est de la complexité en mémoire, chaque agent garde en mémoire une liste des nœuds libres possédant une odeur et une liste des nœuds libres ne possédant pas d'odeur. Soit S la liste des nœuds que l'agent observe, alors au maximum l'agent gardera deux listes de taille $|S|$. De plus, chaque appel à la méthode *getShortestPathWithBlockedNodes* reconstruit un graphe pour éviter les nœuds inaccessibles. D'où la complexité en mémoire dans le pire des cas est en
$O(|S| + |S| + |E| + |V| \times log(|V|))) = O(|S| + |E| + |V| \times log(|V|)))$.
En ce qui concerne la complexité en messages, on en envoie un message à chaque fois qu'on voit un Golem. Ainsi, un agent envoie cette information à tous les agents, la complexité en message est en $O(n)$, avec n le nombre d'agents.

Cette implémentation permet de partager la position du Golem, tout en permettant le suivi et la localisation du Golem. Cependant, le recalcule du chemin vers le Golem peut-être coûteux en temps. Aussi, si un agent reçoit la position d'un Golem, mais que sa carte ne lui permet pas de s'y rendre, alors il abandonne le suivi du Golem.

3.4 Mur

Le comportement mur constitue la phase finale de la chasse. Il est activé lorsque l'agent est à côté du Golem. Il permet notamment de constituer une stratégie d'encerclement collectif. L'objectif n'est plus de poursuivre, mais d'occuper les positions autour du golem afin de l'empêcher de se déplacer.

L'agent commence par vérifier s'il a eu sa position de mur changer en réponse au protocal *MOVE-ASIDE*, dans quel cas il se repositionnera autour du golem.

Puis il observe son environnement et vérifier si le Golem est encore visible. Si le Golem est encore présent autour de l'agent alors on modifie sa position si nécessaire. Si le Golem a disparu, l'agent considère que l'encerclement a échoué et continue en chasse pour le poursuivre.

De plus, l'agent récupère les nœuds voisins au Golem et la position des agents visibles, pour ensuite garder en mémoire les nœuds voisins du Golem qui sont libres. Au final, l'agent diffuse la position du Golem, la liste des nœuds bloqués, ainsi que la liste des nœuds voisins libres autour du Golem. Cette implémentation permet aux autres agents de choisir un nœud utile pour l'encerclement du Golem, tout en évitant les conflits de position.

Aussi, nous avons introduit un chrono qui permet de détecter qu'un Golem est bloqué s'il ne bouge pas pendant un certain temps. Ainsi, chaque agent pourra considérer l'encerclement comme étant réussi.

En ce qui concerne la complexité en nombre de messages envoyé, chaque agent envoie la position du Golem, la liste des nœuds bloqués, ainsi que les nœuds libres au alentour. Donc, notre agent envoie dans le pire des cas $O(n)$ messages, avec n le nombre d'agents. Pour la complexité en temps, l'agent commence par récupérer les nœuds autour du Golem. Ce qui prend $O(|V|+|S|)$ avec V les nœuds du graphe, et S les nœuds voisins du Golem. Ensuite, pour chaque destinataire, l'agent calcule sa présence autour du Golem. Ce qui prend $O(n \times |S|)$. Ensuite, il parcourt les positions des agents autour pour créer la liste des nœuds bloqués en $O(n)$. Au final, dans le pire des cas l'algorithme prend $O(|V|+|S|+n \times |S|+n)$. Enfin, pour la complexité en mémoire, l'agent garde en mémoire plusieurs informations. Les nœuds voisins du Golem prennent $O(|S|)$ de mémoire. La position des agents autour du Golem prend $O(|S|)$ en mémoire. Les voisins libres autour du Golem prennent dans le pire des $O(|S|)$ de mémoire. La liste des nœuds bloquée prend dans le pire des cas $O(|S|)$ de mémoire. Donc, la complexité en mémoire est dans le pire des cas $O(4 \times |S|) = O(|S|)$.

4 Conclusion

Ce projet met en œuvre un système multi-agent capable d'explorer un environnement inconnu, de détecter une entité mobile et de coordonner une stratégie collective pour la capturer. L'architecture repose sur une décomposition en 4 comportements, l'exploration, la communication, la chasse et le mur. Chacun de ces comportements est déclenché à l'aide d'un automate à état finis (FSM).

L'exploration s'appuie sur une stratégie basée sur la découverte de nœuds ouverts et l'utilisation de plus courts chemins. La carte est progressivement enrichie et partagée entre les agents, ce qui a pour but d'améliorer la perception de l'environnement. Le comportement de chasse permet de suivre les traces d'un Golem. Dans cet état, nos agents sont chargés de suivre l'odeur du Golem jusqu'à trouver sa position exacte. Enfin, le comportement mur permet de passer de la poursuite à l'encerclement du Golem. En effet, dans cet état, les agents s'organisent pour bloquer collectivement les nœuds voisins du Golem. Cette phase repose fortement sur la communication et sur la coordination de nos agents.

Malgré ses résultats satisfaisants, plusieurs limites apparaissent. Actuellement, lorsqu'un agent reçoit la position d'un Golem et que ce nœud n'est accessible, il abandonne la poursuite du Golem. Nous pourrions envisager d'envoyer un chemin entre l'agent et le Golem. Ce changement permettrait aux agents de toujours pouvoir poursuivre le Golem. La gestion des conflits peut être améliorée. En effet, plusieurs agents peuvent viser la même cible ou se bloquer mutuellement. Nous avons implémenté des mécanismes pour éviter un maximum les collisions entre agents.

Plusieurs pistes d'amélioration peuvent être envisagées. L'exploration des agents peut être améliorée. En effet, ne autre amélioration intéressante aurait été de partager les nœuds qu'un agent vient de visiter récemment. Ceci permettrait à un agent d'éviter de repasser dans un endroit déjà visité par un autre agent nous aurions pu implémenter un partitionnement du graphe pour éviter que les agents explorent les mêmes zones. Aussi, chaque agent pourrait envoyer son chemin vers un nœud pour éviter en avance les collisions avec d'autres agents. Pour l'encerclement, un agent pourrait annoncer à tous qu'il va bloquer un nœud libre. Ceci aura pour effet de réduire encore plus les collisions. De plus, nous aurions pu créer un système qui permet aux agents de bloquer des nœuds importants. Les nœuds importants seraient des nœuds qui séparent le graphe en deux parties connexe. Pour finir, nous aurions pu ajouter un système qui permet aux agents mur de confirmer que le Golem est bien encerclé. Cette idée pourrait permettre d'éviter que les agents restent en chasse autour du Golem.